

Kode ini adalah sebuah class **EdgeDetectionManager** yang berfungsi sebagai "pusat kontrol" atau *facade* untuk menangani berbagai algoritma deteksi tepi (Edge Detection) dalam aplikasi pengolahan citra (MiniPhotoshop).

Tujuan utama class ini adalah menerima gambar input (Bitmap), mengubahnya menjadi hitam-putih (Grayscale), lalu mengirimnya ke algoritma spesifik (Robert, Canny, Sobel, atau Prewitt) untuk diproses.

Berikut adalah penjelasan detail per baris dan per blok:

1. Import Namespace (Baris 1-5)

C#

```
using System;
using System.Drawing;
using System.Drawing.Imaging;
// PENTING: Panggil namespace folder EdgeDetection agar file Robert.cs, Sobel.cs, dll dikenali
using MiniPhotoshop.Logic.EdgeDetection;
```

- **using System;** Mengimpor fungsi dasar C#.
- **using System.Drawing;** Mengimpor class untuk grafis dasar, seperti Bitmap (untuk memanipulasi gambar pixel per pixel) dan Graphics.
- **using System.Drawing.Imaging;** Mengimpor fitur manipulasi gambar tingkat lanjut, seperti ColorMatrix (digunakan di fungsi Grayscale nanti).
- **using MiniPhotoshop.Logic.EdgeDetection;** **Ini sangat penting.** Baris ini memberi tahu kode ini di mana harus mencari class logika algoritma seperti Robert, Sobel, Canny, dan Prewitt. Tanpa ini, kode tidak akan mengenali class Robert worker = new Robert().

2. Definisi Class (Baris 7-9)

C#

```
namespace MiniPhotoshop.Logic.Helpers
{
```

```
public class EdgeDetectionManager
{
```

- **namespace:** Mengelompokkan class ini ke dalam folder logis Helpers.
 - **public class EdgeDetectionManager:** Mendefinisikan class yang bisa diakses oleh bagian lain program (misalnya dari tombol di Form UI).
-

3. Fungsi Robert (Baris 13-21)

C#

```
public Bitmap ProcessRobert(Bitmap source)
{
    // Langkah 1: Ubah ke Grayscale dulu (Wajib untuk akurasi)
    Bitmap gray = MakeGrayscale(source);

    // Langkah 2: Panggil Worker Robert
    Robert worker = new Robert();
    return worker.Apply(gray);
}
```

- **ProcessRobert(Bitmap source):** Menerima gambar asli (source) dan akan mengembalikan gambar hasil (Bitmap).
 - **MakeGrayscale(source):** Memanggil fungsi helper (dijelaskan di bawah) untuk mengubah gambar berwarna menjadi abu-abu.
 - *Kenapa?* Deteksi tepi bekerja berdasarkan perubahan intensitas cahaya, bukan warna. Mengubah ke grayscale menyederhanakan perhitungan dari 3 channel (R,G,B) menjadi 1 channel intensitas.
 - **new Robert():** Membuat *instance* (objek baru) dari class Robert (yang ada di namespace EdgeDetection).
 - **worker.Apply(gray):** Menjalankan rumus matematika Robert Cross pada gambar abu-abu tersebut dan mengembalikan hasilnya.
-

4. Fungsi Canny (Baris 26-37)

C#

```

public Bitmap ProcessCanny(Bitmap source)
{
    if (source == null) return null;

    Bitmap gray = MakeGrayscale(source);

    // Low: 5f (Sangat sensitif)
    // High: 20f (Cukup longgar)
    Canny worker = new Canny(5f, 20f);

    return worker.Apply(gray);
}

```

- **if (source == null):** Pengecekan keamanan (validasi) agar program tidak *crash* jika gambar kosong.
- **new Canny(5f, 20f):** Membuat objek algoritma Canny dengan dua parameter **Threshold**:
 - **5f (Low Threshold):** Batas bawah. Garis tepi yang lemah tapi tersambung dengan garis kuat akan tetap dianggap garis tepi.
 - **20f (High Threshold):** Batas atas. Garis tepi yang sangat tegas (perbedaan warna kontras) pasti diambil.
 - Angka ini menentukan seberapa "cerewet" atau sensitif deteksinya.

5. Fungsi Sobel (Baris 42-52)

C#

```

public Bitmap ProcessSobel(Bitmap source)
{
    if (source == null) return null;

    // 1. Ubah ke Grayscale (Sobel bekerja di 1 channel warna)
    Bitmap gray = MakeGrayscale(source);

    // 2. Panggil Worker Sobel
    Sobel worker = new Sobel();
    return worker.Apply(gray);
}

```

- Sama seperti Robert, namun menggunakan class **Sobel**. Sobel biasanya lebih halus daripada Robert karena menggunakan matriks kernel 3x3, sedangkan Robert hanya 2x2.

6. Fungsi Prewitt (Baris 57-68)

C#

```
public Bitmap ProcessPrewitt(Bitmap source)
{
    if (source == null) return null;

    Bitmap gray = MakeGrayscale(source);

    // 2. Panggil worker Prewitt
    // Menggunakan full namespace ...
    MiniPhotoshop.Logic.EdgeDetection.Prewitt worker = new
MiniPhotoshop.Logic.EdgeDetection.Prewitt();
    return worker.Apply(gray);
}
```

- **MiniPhotoshop.Logic.EdgeDetection.Prewitt:** Di sini penulis kode menuliskan nama *namespace* secara lengkap (eksplisit).
 - *Tujuannya:* Kadang dilakukan untuk menghindari kebingungan jika ada class lain yang juga bernama Prewitt. Namun, karena di atas sudah ada using, sebenarnya cukup ditulis new Prewitt() saja. Kode ini tetap valid.

7. Helper: Grayscale Converter (Baris 73-93)

Ini adalah bagian paling teknis secara matematika di file ini.

C#

```
private Bitmap MakeGrayscale(Bitmap original)
{
    // Buat kanvas kosong seukuran gambar asli
```

```
Bitmap newBmp = new Bitmap(original.Width, original.Height);
```

```
// Buat objek Graphics untuk menggambar di kanvas baru
```

```
using (Graphics g = Graphics.FromImage(newBmp))
```

```
{
```

```
    // Definisikan Matriks Warna (Color Matrix)
```

```
    ColorMatrix colorMatrix = new ColorMatrix(
```

```
        new float[][]
```

```
        {
```

```
            // Baris Merah (Red): Diambil 29.9%
```

```
            new float[] { .299f, .299f, .299f, 0, 0 },
```

```
            // Baris Hijau (Green): Diambil 58.7%
```

```
            new float[] { .587f, .587f, .587f, 0, 0 },
```

```
            // Baris Biru (Blue): Diambil 11.4%
```

```
            new float[] { .114f, .114f, .114f, 0, 0 },
```

```
            // Alpha (Transparansi): Tetap 100%
```

```
            new float[] { 0, 0, 0, 1, 0 },
```

```
            // Translasi (Geser warna): 0
```

```
            new float[] { 0, 0, 0, 0, 1 }
```

```
        });
```

- **Logika ColorMatrix:**

- Alih-alih melakukan *looping* manual tiap pixel (yang lambat), kode ini menggunakan **ColorMatrix** dari GDI+ yang sangat cepat.
- Angka **0.299**, **0.587**, **0.114** adalah standar internasional (Rec. 601) untuk persepsi mata manusia terhadap terang-gelap.
- Mata manusia paling sensitif terhadap warna Hijau (Green), makanya nilainya paling besar (0.587 atau 58%).
- Rumusnya: $\$PixelBaru = (R \times 0.299) + (G \times 0.587) + (B \times 0.114)\$$.

C#

```
ImageAttributes attributes = new ImageAttributes();
```

```
attributes.SetColorMatrix(colorMatrix);
```

```
// Gambar ulang foto asli ke kanvas baru dengan menerapkan filter matriks di atas
```

```
g.DrawImage(original, new Rectangle(0, 0, original.Width, original.Height),
```

```
    0, 0, original.Width, original.Height, GraphicsUnit.Pixel, attributes);
```

```
}
```

```
return newBmp;
```

```
}
```

- **attributes.SetColorMatrix:** Memasukkan rumus matematika tadi ke dalam pengaturan gambar.
- **g.DrawImage(...):** Ini adalah perintah eksekusinya. "Gambar original ke dalam newBmp, tapi saat menggambar, terapkan attributes (rumus grayscale) tadi."

Kesimpulan untuk Anda

Kode ini sudah **siap pakai** sebagai penghubung (Manager). Anda tidak perlu mengubah logika di dalam MakeGrayscale.

Tugas Anda selanjutnya (berdasarkan komentar di kode) adalah memastikan class Robert, Canny, Sobel, dan Prewitt yang dipanggil di sini sudah terimplementasi dengan benar di file masing-masing dalam folder EdgeDetection. Class EdgeDetectionManager ini hanya bertugas memanggil mereka.

Apakah Anda ingin saya menjelaskan isi dari salah satu algoritma tersebut (misalnya, bagaimana cara kerja Robert.cs)?